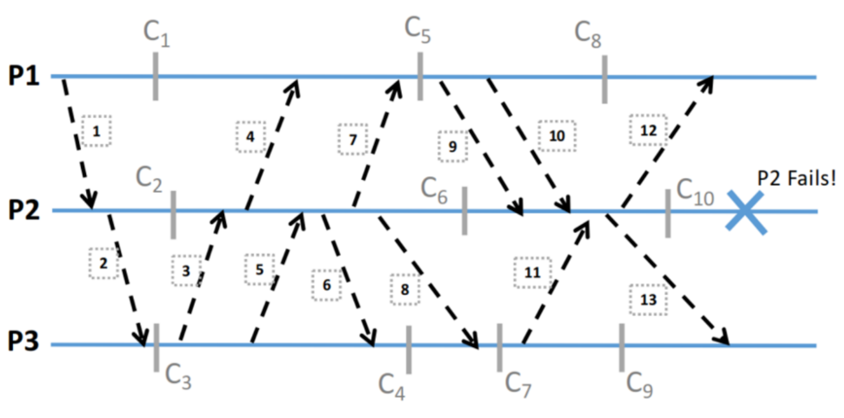
Problem 1

You are building a new distributed application where three nodes (P1, P2, P3) communicate by exchanging messages (shown in square boxes). Ideally, you wish to ensure that each node has same view of the shared state of the app. However, you understand the reality of distributed systems and know machines may go down. Nevertheless, the key is to be able to recover from these failures and bring each machine back to a consistent state. To do so, you implement a checkpointing solution where each node takes uncoordinated and independent snapshot of their state of the world (C1 - C10 in the example). Now,



(a) Process P2 fails. Can you recover from this failure and get your distributed system back to a consistent state using the different checkpoints you have saved? In particular, calculate a consistent recovery line (listing the snapshots) for rollback.

Note:

- *If there is no recovery line state why.*

- *If there are multiple, list all of them stating which one is ideal. Explain your choice.*

(b) Explain how you could use logging and replay to augment this checkpointed system to do even better recovery while being more effcient in terms of a checkpoint only solution.

Problem 2

Consider the state of a RAFT system on five servers, S1—S5.

S1	1	1	3	
S2	1	1	2	4
S3	1			
S4	1	1	2	
S5	1	1	2	

Here, each cell represents a log entry. The number in the log entry indicates the term number.
(a) Which of the following are consistent with this log? Circle all that apply.

- *S1 was the leader in term 3; S1, S3, and S4 voted for S1 in term 3.*
- *S1 was the leader in term 3; S1, S2, and S4 voted for S1 in term 3.*
- *S3 was the leader in term 1.*
- *S2 was the leader in term 2; S3, S4, and S5 voted for S2 in term 2.*
- *S5 was the leader in term 2; S2, S4, and S5 received the log entry for term 2 before term 3 began.*

(b) If the system runs for a while, which of of the following are possible prefixes for the committed state of the log? Circle all that apply.

S1	1	1	3
S2	1	1	3
S3	1	1	3
S4	1	1	3
S5	1	1	3

(a)

S1	1	1	4
S2	1	1	4
S3	1	1	4
S4	1	1	4
S5	1	1	4

(b)

S1	1	1	2	4
S2	1	1	2	4
S3	1	1	2	4
S4	1	1	2	4
S5	1	1	2	4

(c)

Problem 3

Answer each of these questions with reference to the Raft protocol described in the Raft paper's Figure2.

(a) Suppose the leader waited for all servers (rather than just a majority) to have matchIndex[i]   N before setting commitIndex to N (at the end of Rules for Servers). What specific valuable property of Raft would this change break?

(b) Suppose the leader immediately advanced commitIndex to its last log entry without waiting for responses to AppendEntries RPCs, and regardless of the values inmatchIndex. Describe a specific situation in which this change would lead to incorrect execution of the service being replicated by Raft.

(c) There are five Raft servers. S1 is the leader in term 17. S1 and S2 become isolated in a network partition. S3, S4, and S5 choose S3 as the leader for term 18. S3, S4,and S5 commit and execute commands in term 18. S1 crashes and restarts, and S1 and S2 run many elections, so that their currentTerms are 50. S3 crashes but does not re-start, and an instant later the network partition heals, so that S1, S2, S4, and S5 can communicate. Could S1 become the next leader? How could that happen, or what prevents it from happening? Your answer should refer to specific rules in the Raft paper, and explain how the rules apply in this specific scenario.

Problem 4

Investigate the blocking problem in the 2-Phase Commit (2PC) protocol. Blocking means that a participant in the protocol cannot decide whether to commit or abort a transaction. Provide an example scenario in which this blocking problem occurs and explain why the participant is unable to make progress.